Міністерство освіти і науки України
Національний університет «Одеська політехніка»
Навчально-науковий інститут комп’ютерних систем
Кафедра інформаційних систем

Лабораторна робота № 2
з дисципліни: «Теорія алгоритмів»
Тема: «Логарифмічні алгоритми сортування»

Варіант № 22

Виконав:
Студент групи АІ-254
Церковний О. Г.
Перевірили:
Смик С. Ю.
Арсірій О.О.

Одеса 2026

Мета роботи: Метою виконання лабораторної роботи є набуття практичних навичок із проектування, реалізації, тестування та аналізу алгоритмів сортування злиттям ітеративної та рекурсивної версії, а також рекурсивного швидкого сортування. За рахунок виконання трасування та візуалізації у вигляді графа. Студент має практично довести логарифмічну часову складність та стійкість або нестійкість вивчених алгоритмів, а також порівняти ці алгоритми за кількістю базових операцій.

Завдання:
1. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм Сортування злиттям.
2. Спроектувати, реалізувати, протестувати та проаналізувати алгоритм швидкого сортування
3. Порівняти обчислювальну складність алгоритмів сортування злиттям та швидкого за базовими операціями.
Варіант 22. Послідовність: 19, 75, 43, 31, 5, 66, 62, 34, 76

1. Сортування злиттям

1.1 Ітеративна та рекурсивна версії алгоритму

Таблиця 2.1 - Псевдокоди ітеративного та рекурсивного алгоритму сортування злиттям

Ітеративний алгоритм	Рекурсивний алгоритм
function mergeSortIterative(a : int[n]): for i = 1 to n, i *= 2 for j = 0 to n - i, j += 2 * i merge_iterative(a, j, j + i, min(j + 2 * i, n))	MergeSortRecursive(A) n ← length(A) if n ≤ 1 then return A mid ← n // 2 left_half ← A[0...mid-1] right_half ← A[mid...n-1] sorted_left ← MergeSortRecursive(left_half) sorted_right ← MergeSortRecursive(right_half) return Merge(sorted_left, sorted_right)
function merge_iterative(a : int[n]; left, mid, right : int): it1 = 0 it2 = 0 result : int[right - left] while left + it1 < mid and mid + it2 < right if a[left + it1] <= a[mid + it2]	Merge(Left, Right): Result ← [] i ← 0 j ← 0 while i < length(Left) and j < length(Right) do if Left[i] ≤ Right[j] then append Left[i] to Result

```
        result[it1 + it2] = a[left + it1]
        it1 += 1
      else
        result[it1 + it2] = a[mid + it2]
        it2 += 1

    while left + it1 < mid
      result[it1 + it2] = a[left + it1]
      it1 += 1

    while mid + it2 < right
      result[it1 + it2] = a[mid + it2]
      it2 += 1

    for i = 0 to it1 + it2
      a[left + i] = result[i]
```

```
        i ← i + 1
      else
        append Right[j] to Result
        j ← j + 1

    while i < length(Left) do
      append Left[i] to Result
      i ← i + 1

    while j < length(Right) do
      append Right[j] to Result
      j ← j + 1

    return Result
```

Ітеративний алгоритм сортування злиттям послідовно об'єднує відсортовані підмасиви, поступово збільшуючи їхній розмір удвічі. Рекурсивний алгоритм розділяє початковий масив на дві частини, доки кожна частина не міститиме одного елемента, після чого об'єднує частини у відсортованому порядку. Обидва алгоритми використовують принцип «розділяй і володарюй».

1.2 Реалізація ітеративної та рекурсивної версій алгоритму сортування злиттям мовою Python

Лістинг 2.1 – Python-код ітеративної реалізації алгоритму сортування злиттям

```
def merge_sort_iterative(a):
    n = len(a)
    comparisons = 0
    assignments = 0
    i = 1

    while i < n:
        j = 0

        while j < n - i:
            left = j
            mid = j + i
            right = min(j + 2 * i, n)

            c, a_count = merge(a, left, mid, right)
            comparisons += c
            assignments += a_count

            j += 2 * i

        i *= 2

    return a, comparisons, assignments
```

```
def merge(a, left, mid, right):
    comparisons = 0
    assignments = 0

    n1 = mid - left
    n2 = right - mid

    L = a[left:mid]
    R = a[mid:right]
    assignments += n1 + n2

    it1 = 0
    it2 = 0
    k = left
    assignments += 3

    while it1 < n1 and it2 < n2:
        comparisons += 1

        if L[it1] <= R[it2]:
            a[k] = L[it1]
            it1 += 1
            assignments += 1
        else:
            a[k] = R[it2]
            it2 += 1
            assignments += 1

        k += 1
        assignments += 1

    while it1 < n1:
        a[k] = L[it1]
        it1 += 1
        k += 1
        assignments += 1

    while it2 < n2:
        a[k] = R[it2]
        it2 += 1
        k += 1
        assignments += 1

    return comparisons, assignments


# Перевірка роботи алгоритму
my_list = [19, 75, 43, 31, 5, 66, 62, 34, 76]

print("Оригінальний список:", my_list)
sorted_list, comps, assigs = merge_sort_iterative(my_list.copy())

print("Відсортований список:", sorted_list)
print("Кількість порівнянь:", comps)
print("Кількість присвоювань:", assigs)
```

Лістинг 2.2 – Python-код рекурсивної реалізації алгоритму сортування злиттям.

```
def merge_sort_recursive(arr):
    comparisons = 0
    assignments = 0
    recursive_calls = 0
```

```
    if len(arr) <= 1:
        return arr, comparisons, assignments, recursive_calls

    mid = len(arr) // 2
    assignments += 1

    left_half, c1, a1, r1 = merge_sort_recursive(arr[:mid])
    right_half, c2, a2, r2 = merge_sort_recursive(arr[mid:])

    comparisons += c1 + c2
    assignments += a1 + a2
    recursive_calls += r1 + r2 + 2

    merged_arr, c_merge, a_merge = merge(left_half, right_half)

    comparisons += c_merge
    assignments += a_merge

    return merged_arr, comparisons, assignments, recursive_calls


def merge(left, right):
    merged_arr = []
    comparisons = 0
    assignments = 0
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        comparisons += 1

        if left[i] <= right[j]:
            merged_arr.append(left[i])
            i += 1
        else:
            merged_arr.append(right[j])
            j += 1

        assignments += 1

    while i < len(left):
        merged_arr.append(left[i])
        i += 1
        assignments += 1

    while j < len(right):
        merged_arr.append(right[j])
        j += 1
        assignments += 1

    return merged_arr, comparisons, assignments


# Приклад використання
my_list = [19, 75, 43, 31, 5, 66, 62, 34, 76]

print("Оригінальний список:", my_list)

sorted_list, total_comparisons, total_assignments, total_recursive_calls = \
    merge_sort_recursive(my_list)

print("Відсортований список:", sorted_list)
print("Загальна кількість порівнянь:", total_comparisons)
```

```
print("Загальна кількість присвоювань:", total_assignments)
print("Загальна кількість рекурсивних викликів:", total_recursive_calls)
```

Результати виконання Python-коду, який наведено в Лістінг 2.1 та Лістінг Трасування Python-коду2.2 показано в таблиці 2.2

Таблиця 2.2 Результати ітеративної та рекурсивної реалізації алгоритму сортування злиттям

Ітеративний алгоритм	Рекурсивний алгоритм
Оригінальний список: [19, 75, 43, 31, 5, 66, 62, 34, 76] Відсортований список: [5, 19, 31, 34, 43, 62, 66, 75, 76] Кількість порівнянь: 25 Кількість присвоювань: 115	Оригінальний список: [19, 75, 43, 31, 5, 66, 62, 34, 76] Відсортований список: [5, 19, 31, 34, 43, 62, 66, 75, 76] Загальна кількість порівнянь: 21 Загальна кількість присвоювань: 37 Загальна кількість рекурсивних викликів: 16

1.3 Трасування ітеративної та рекурсивної версій Python-коду алгоритму сортування злиттям

Таблиця 2.3 Результати трасування алгоритму сортування злиттям.

Трасування Python-коду Лістінг 2.1	Трасування Python-коду Лістінг 2.2
--- ІТЕРАТИВНА ВЕРСІЯ --- Початковий масив: [19, 75, 43, 31, 5, 66, 62, 34, 76] ---------- Об'єднуємо підмасиви: [19] і [75] Порівняння: 19 <= 75 Масив після об'єднання: [19, 75, 43, 31, 5, 66, 62, 34, 76] ---------- Об'єднуємо підмасиви: [43] і [31] Порівняння: 43 <= 31 Масив після об'єднання: [19, 75, 31, 43, 5, 66, 62, 34, 76] ---------- Об'єднуємо підмасиви: [5] і [66] Порівняння: 5 <= 66 Масив після об'єднання: [19, 75, 31, 43, 5, 66, 62, 34, 76] ---------- Об'єднуємо підмасиви: [62] і [34] Порівняння: 62 <= 34 Масив після об'єднання: [19, 75, 31, 43, 5, 66, 34, 62, 76]	--- РЕКУРСИВНА ВЕРСІЯ --- Розділяємо масив: [19, 75, 43, 31, 5, 66, 62, 34, 76] Розділяємо масив: [19, 75, 43, 31] Розділяємо масив: [19, 75] Розділяємо масив: [19] Розділяємо масив: [75] Зливаємо [19] та [75] Порівняння: 19 <= 75 -> True. Додаємо 19 Додаємо залишок з правого масиву: 75 Злиття завершено. Результат: [19, 75] Розділяємо масив: [43, 31] Розділяємо масив: [43] Розділяємо масив: [31] Зливаємо [43] та [31] Порівняння: 43 <= 31 -> False. Додаємо 31 Додаємо залишок з лівого масиву: 43 Злиття завершено. Результат: [31, 43] Зливаємо [19, 75] та [31, 43] Порівняння: 19 <= 31 -> True. Додаємо 19 Порівняння: 75 <= 31 -> False. Додаємо 31

```
----------------------------------------
Об'єднуємо підмасиви: [19, 75] і [31, 43]
Порівняння:        19        <=        31
Порівняння:        75        <=        31
Порівняння:        75        <=        43
Масив після об'єднання: [19, 31, 43, 75, 5, 66, 34, 62, 76]
----------------------------------------
Об'єднуємо підмасиви: [5, 66] і [34, 62]
Порівняння:        5        <=        34
Порівняння:        66       <=        34
Порівняння:        66       <=        62
Масив після об'єднання: [19, 31, 43, 75, 5, 34, 62, 66, 76]
----------------------------------------
Об'єднуємо підмасиви: [19, 31, 43, 75] і [5,         34,         62,         66]
Порівняння:        19        <=        5
Порівняння:        19        <=        34
Порівняння:        31        <=        34
Порівняння:        43        <=        34
Порівняння:        43        <=        62
Порівняння:        75        <=        62
Порівняння:        75        <=        66
Масив після об'єднання: [5, 19, 31, 34, 43, 62, 66, 75, 76]
----------------------------------------
Об'єднуємо підмасиви: [5, 19, 31, 34, 43,     62,     66,     75]     і     [76]
Порівняння:        5        <=        76
Порівняння:        19       <=        76
Порівняння:        31       <=        76
Порівняння:        34       <=        76
Порівняння:        43       <=        76
Порівняння:        62       <=        76
Порівняння:        66       <=        76
Порівняння:        75       <=        76
Масив після об'єднання: [5, 19, 31, 34, 43, 62, 66, 75, 76]
----------------------------------------
Фінальний відсортований список: [5, 19, 31,   34,   43,   62,   66,   75,   76]
Загальна   кількість   порівнянь:   25
Загальна кількість присвоювань: 115
```

```
Порівняння: 75 <= 43 -> False. Додаємо 43
Додаємо залишок з лівого масиву: 75
Злиття завершено. Результат: [19, 31, 43, 75]
Розділяємо масив: [5, 66, 62, 34, 76]
Розділяємо      масив:      [5,      66]
Розділяємо          масив:          [5]
Розділяємо          масив:          [66]
Зливаємо        [5]        та        [66]
Порівняння: 5 <= 66 -> True. Додаємо 5
Додаємо залишок з правого масиву: 66
Злиття завершено. Результат: [5, 66]
Розділяємо   масив:   [62,   34,   76]
Розділяємо          масив:          [62]
Розділяємо      масив:      [34,      76]
Розділяємо          масив:          [34]
Розділяємо          масив:          [76]
Зливаємо        [34]        та        [76]
Порівняння: 34 <= 76 -> True. Додаємо 34
Додаємо залишок з правого масиву: 76
Злиття завершено. Результат: [34, 76]
Зливаємо    [62]    та    [34,    76]
Порівняння: 62 <= 34 -> False. Додаємо 34
Порівняння: 62 <= 76 -> True. Додаємо 62
Додаємо залишок з правого масиву: 76
Злиття завершено. Результат: [34, 62, 76]
Зливаємо [5, 66] та [34, 62, 76]
Порівняння: 5 <= 34 -> True. Додаємо 5
Порівняння: 66 <= 34 -> False. Додаємо 34
Порівняння: 66 <= 62 -> False. Додаємо 62
Порівняння: 66 <= 76 -> True. Додаємо 66
Додаємо залишок з правого масиву: 76
Злиття завершено. Результат: [5, 34, 62, 66, 76]
Зливаємо [19, 31, 43, 75] та [5, 34, 62, 66,                                  76]
Порівняння: 19 <= 5 -> False. Додаємо 5
Порівняння: 19 <= 34 -> True. Додаємо 19
Порівняння: 31 <= 34 -> True. Додаємо 31
Порівняння: 43 <= 34 -> False. Додаємо 34
Порівняння: 43 <= 62 -> True. Додаємо 43
Порівняння: 75 <= 62 -> False. Додаємо 62
Порівняння: 75 <= 66 -> False. Додаємо 66
Порівняння: 75 <= 76 -> True. Додаємо 75
Додаємо залишок з правого масиву: 76
Злиття завершено. Результат: [5, 19, 31, 34, 43, 62, 66, 75, 76]
Фінальний відсортований список: [5, 19, 31,   34,   43,   62,   66,   75,   76]
Загальна   кількість   порівнянь:   21
Загальна   кількість   присвоювань:   37
Загальна кількість рекурсивних викликів: 16
```

Під час трасування ітеративної версії масив послідовно об'єднується у відсортовані підмасиви розміром 1, 2, 4 та 8 елементів. У рекурсивній версії масив спочатку ділиться на менші частини до отримання підмасивів з одним елементом, після чого вони об'єднуються у відсортованому порядку. Обидві реалізації отримали однаковий результат: [5, 19, 31, 34, 43, 62, 66, 75, 76].

На рисунку 1 показано результати візуалізації трасування ітеративної (Рис. 2.1,а) та рекурсивної (Рис. 2,б) версій Python-коду алгоритму сортування злиттям.

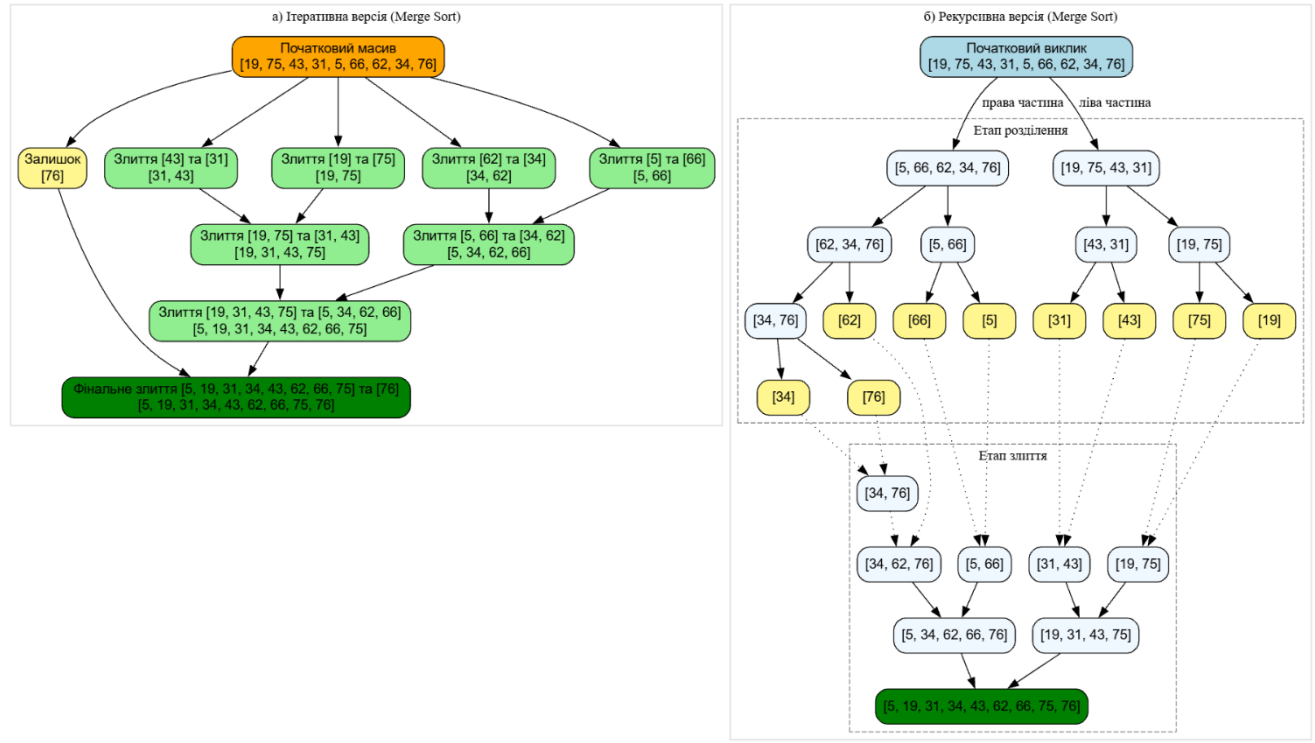


Рисунок 2.1 – Результати візуалізації трасування ітеративної та рекурсивної версій Python-коду алгоритму сортування злиттям

На графі показано процес розділення початкового масиву на підмасиви та їх подальшого об'єднання у відсортованому порядку. Рекурсивна версія послідовно розділяє масив на менші частини, а потім об'єднує їх. Ітеративна версія виконує об'єднання підмасивів зі збільшенням їхнього розміру на кожному етапі. Обидві реалізації забезпечують однаковий відсортований результат.

2. Швидке сортування

2.1 Рекурсивна версія алгоритму швидкого сортування

Таблиця 2.4 Приклади псевдокоду рекурсивної процедури та процедури розбиття алгоритму швидкого сортування за схемою Хоара

Рекурсивна процедура quicksort quicksort(a, l, r)	Процедура розбиття partition

quicksort(a, l, r)
if l <= r:
  q = partition(a, l, r)
 	
  quicksort(a, l, q)
  // Рекурсивний виклик для лівого підмасиву
 	
  quicksort(a, q + 1, r)
  // Рекурсивний виклик для правого підмасиву
	partition(a, l, r)
	pivot = a[l]
	i = l - 1
	j = r + 1
	
	while True:
	  i += 1
	  while a[i] <= pivot:
	  i += 1
	
	  j -= 1
	  while a[j] > pivot:
	  j -= 1
	
	  if i >= j:
	  return j
	
	  a[i], a[j] = a[j], a[i]

2.2 Реалізація алгоритму швидкого сортування мовою Python

Лістинг 2.3 – Python-код рекурсивної реалізації алгоритму швидкого сортування за схемою Хоара

```
def quicksort(a, l, r):
    comparisons = 0
    assignments = 0
    recursive_calls = 1

    if l <= r:
        q, c1, a1 = partition(a, l, r)
        comparisons += c1
        assignments += a1

        c2, a2, r2 = quicksort(a, l, q)
        c3, a3, r3 = quicksort(a, q + 1, r)

        comparisons += c2 + c3
        assignments += a2 + a3
        recursive_calls += r2 + r3
    else:
        return 0, 0, 0

    return comparisons, assignments, recursive_calls


def partition(a, l, r):
    comparisons = 0
    assignments = 0

    pivot = a[l]
```

```
    assignments += 1

    i = l - 1
    j = r + 1
    assignments += 2

    while True:
        i += 1
        assignments += 1

        while a[i] <= pivot:
            comparisons += 1
            i += 1
            assignments += 1
        comparisons += 1

        j -= 1
        assignments += 1

        while a[j] > pivot:
            comparisons += 1
            j -= 1
            assignments += 1
        comparisons += 1

        if i >= j:
            return j, comparisons, assignments

        a[i], a[j] = a[j], a[i]
        assignments += 3


# Приклад використання
my_list = [19, 75, 43, 31, 5, 66, 62, 34, 76]

print("Оригінальний список:", my_list)

comparisons, assignments, recursive_calls = quicksort(
    my_list, 0, len(my_list) - 1
)

print("Відсортований список:", my_list)
print("Загальна кількість порівнянь:", comparisons)
print("Загальна кількість присвоювань:", assignments)
print("Загальна кількість рекурсивних викликів:", recursive_calls)
```

Примітка: лічильник recursive_calls враховує лише виклики, що виконують розбиття (l <= r), їх 8. Загальна кількість викликів quicksort, включно з тривіальними (l ≥ r, масив з одного елемента), дорівнює 17 ($2n - 1$ при $n = 9$), що відповідає кількості рядків «Quicksort виклик» у трасуванні та вузлів на графі (рис. 2.2).

Результати виконання Python-коду, який наведено в Лістінг 2.3 показано далі:

Оригінальний список: [19, 75, 43, 31, 5, 66, 62, 34, 76]
Відсортований список: [5, 19, 31, 34, 43, 62, 66, 75, 76]
Загальна кількість порівнянь: 49
Загальна кількість присвоювань: 91

Загальна кількість рекурсивних викликів: 8

2.3 Трасування рекурсивної версії Python-коду алгоритму швидкого сортування за схемою Хоара

Таблиця 2.5 Результати трасування алгоритму швидкого сортування

Трасування Python-коду (Лістінг 2.3)

```
--- Quicksort за схемою Хоара з трасуванням ---
Оригінальний список: [19, 75, 43, 31, 5, 66, 62, 34, 76]
Quicksort виклик: масив = [19, 75, 43, 31, 5, 66, 62, 34, 76], l = 0, r = 8
Вибираємо опорний елемент (pivot): 19
Поточні індекси: i = 0, j = 4. Обмінюємо a[0] (19) і a[4] (5).
Масив: [5, 75, 43, 31, 19, 66, 62, 34, 76]
Поточні індекси: i = 1, j = 0. Масив: [5, 75, 43, 31, 19, 66, 62, 34, 76]
Індекси перетнулися. Поділ завершено. Повертаємо j=0.
Quicksort виклик: масив = [5, 75, 43, 31, 19, 66, 62, 34, 76], l = 0, r = 0
Quicksort виклик: масив = [5, 75, 43, 31, 19, 66, 62, 34, 76], l = 1, r = 8
Вибираємо опорний елемент (pivot): 75
Поточні індекси: i = 1, j = 7. Обмінюємо a[1] (75) і a[7] (34).
Масив: [5, 34, 43, 31, 19, 66, 62, 75, 76]
Поточні індекси: i = 7, j = 6. Масив: [5, 34, 43, 31, 19, 66, 62, 75, 76]
Індекси перетнулися. Поділ завершено. Повертаємо j=6.
Quicksort виклик: масив = [5, 34, 43, 31, 19, 66, 62, 75, 76], l = 1, r = 6
  Вибираємо опорний елемент (pivot): 34
  Поточні індекси: i = 1, j = 4. Обмінюємо a[1] (34) і a[4] (19).
  Масив: [5, 19, 43, 31, 34, 66, 62, 75, 76]
  Поточні індекси: i = 2, j = 3. Обмінюємо a[2] (43) і a[3] (31).
  Масив: [5, 19, 31, 43, 34, 66, 62, 75, 76]
  Поточні індекси: i = 3, j = 2. Масив: [5, 19, 31, 43, 34, 66, 62, 75, 76]
  Індекси перетнулися. Поділ завершено. Повертаємо j=2.

  Quicksort виклик: масив = [5, 19, 31, 43, 34, 66, 62, 75, 76], l = 1, r = 2
    Вибираємо опорний елемент (pivot): 19
    Поточні індекси: i = 1, j = 1. Масив: [5, 19, 31, 43, 34, 66, 62, 75, 76]
    Індекси перетнулися. Поділ завершено. Повертаємо j=1.
    Quicksort виклик: масив = [5, 19, 31, 43, 34, 66, 62, 75, 76], l = 1, r = 1
    Quicksort виклик: масив = [5, 19, 31, 43, 34, 66, 62, 75, 76], l = 2, r = 2
  Після Quicksort для l=1, r=2: масив = [5, 19, 31, 43, 34, 66, 62, 75, 76]

  Quicksort виклик: масив = [5, 19, 31, 43, 34, 66, 62, 75, 76], l = 3, r = 6
    Вибираємо опорний елемент (pivot): 43
    Поточні індекси: i = 3, j = 4. Обмінюємо a[3] (43) і a[4] (34).
    Масив: [5, 19, 31, 34, 43, 66, 62, 75, 76]
    Поточні індекси: i = 4, j = 3. Масив: [5, 19, 31, 34, 43, 66, 62, 75, 76]
    Індекси перетнулися. Поділ завершено. Повертаємо j=3.

    Quicksort виклик: масив = [5, 19, 31, 34, 43, 66, 62, 75, 76], l = 3, r = 3
    Quicksort виклик: масив = [5, 19, 31, 34, 43, 66, 62, 75, 76], l = 4, r = 6
      Вибираємо опорний елемент (pivot): 43
      Поточні індекси: i = 4, j = 4. Масив: [5, 19, 31, 34, 43, 66, 62, 75, 76]
      Індекси перетнулися. Поділ завершено. Повертаємо j=4.

      Quicksort виклик: масив = [5, 19, 31, 34, 43, 66, 62, 75, 76], l = 4, r = 4
      Quicksort виклик: масив = [5, 19, 31, 34, 43, 66, 62, 75, 76], l = 5, r = 6
        Вибираємо опорний елемент (pivot): 66
        Поточні індекси: i = 5, j = 6. Обмінюємо a[5] (66) і a[6] (62).
        Масив: [5, 19, 31, 34, 43, 62, 66, 75, 76]
        Поточні індекси: i = 6, j = 5. Масив: [5, 19, 31, 34, 43, 62, 66, 75, 76]
        Індекси перетнулися. Поділ завершено. Повертаємо j=5.

      Quicksort виклик: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76], l = 5, r = 5
      Quicksort виклик: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76], l = 6, r = 6
```

```
      Після Quicksort для l=5, r=6: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]
    Після Quicksort для l=4, r=6: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]
  Після Quicksort для l=3, r=6: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]
Після Quicksort для l=1, r=6: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]

Quicksort виклик: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76], l = 7, r = 8
  Вибираємо опорний елемент (pivot): 75
  Поточні індекси: i = 7, j = 7. Масив: [5, 19, 31, 34, 43, 62, 66, 75, 76]
  Індекси перетнулися. Поділ завершено. Повертаємо j=7.
  Quicksort виклик: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76], l = 7, r = 7
  Quicksort виклик: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76], l = 8, r = 8
Після Quicksort для l=7, r=8: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]
Після Quicksort для l=1, r=8: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]
Після Quicksort для l=0, r=8: масив = [5, 19, 31, 34, 43, 62, 66, 75, 76]
Фінальний відсортований список: [5, 19, 31, 34, 43, 62, 66, 75, 76]
Загальна кількість порівнянь: 49
Загальна кількість присвоювань: 91
Загальна кількість рекурсивних викликів: 8
```

На рисунку2.2 показано результати візуалізації трасування рекурсивної версії Python-коду алгоритму швидкого сортування.

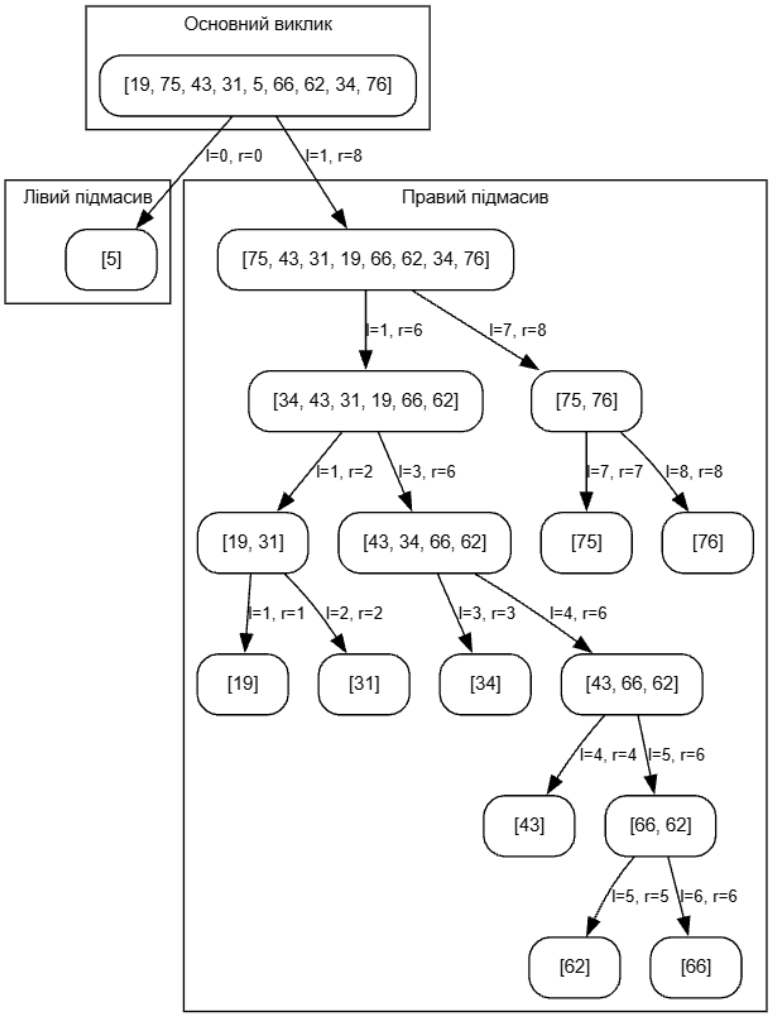


Рисунок 2.2 – Результати візуалізації трасування Python-коду рекурсивного алгоритму швидкого сортування

На рисунку 2.2 показано процес трасування рекурсивного алгоритму швидкого сортування за схемою Хоара. На кожному етапі вибирається опорний елемент, після чого індекси переміщуються назустріч один одному та виконуються обміни елементів. Після завершення розбиття алгоритм рекурсивно обробляє лівий і правий підмасиви. У результаті отримуємо відсортований масив [5, 19, 31, 34, 43, 62, 66, 75, 76]. Усього на графі 17 вузлів (викликів), з яких 8 виконують розбиття (l <= r), а 9 є тривіальними листками, тому recursive_calls = 8.

3. Аналіз обчислювальної складності алгоритмів сортування злиттям та швидкого сортування

Таблиця 3.1 – Порівняння обчислювальної складності алгоритмів сортування злиттям та швидкого сортування

Назва алгоритму	Теоретичні відомості: порівняння	Теоретичні відомості: присвоювання	Результати експерименту: порівняння	Результати експерименту: присвоювання
Merge Sort (ітеративний)	O(n log n)	O(n log n)	25	115
Merge Sort (рекурсивний	O(n log n)	O(n log n)	21	37
Quick Sort	Середній: O(n log n); найгірший: $O(n^2)$	Середній: O(n log n); найгірший: $O(n^2)$	49	91

Висновок: У ході лабораторної роботи № 2 було спроектовано, реалізовано та протестовано ітеративну й рекурсивну версії сортування злиттям, а також рекурсивне швидке сортування за схемою Хоара мовою Python на послідовності варіанта 22 (n = 9). Усі три реалізації дали однаковий результат [5, 19, 31, 34, 43, 62, 66, 75, 76]. Трасування та графи підтвердили принцип «розділяй і володарюй»: злиття розбиває масив навпіл та зливає відсортовані частини (глибина дерева $\lceil \log_2 n \rceil = 4$), а швидке сортування розбиває масив за опорним елементом без етапу злиття. Експериментально отримано: ітеративне злиття — 25 порівнянь та 115 присвоювань, рекурсивне злиття — 21 порівняння, 37 присвоювань і 16 рекурсивних викликів, швидке сортування — 49 порівнянь, 91 присвоювань і 8 викликів із розбиттям. Різниця в присвоюваннях між версіями злиття пояснюється тим, що ітеративна версія враховує копіювання у тимчасові масиви. Швидке сортування виконало більше порівнянь, оскільки в розбитті за Хоара підраховуються й перевірки виходу з циклів, а опорний елемент a[l] дає

незбалансований поділ на цьому наборі. Сортування злиттям має часову складність O(n log n) у всіх випадках, є стійким (при <=2.3 Трасування рекурсивної **версій** Python-коду у функції злиття) та потребує O(n) додаткової пам'яті. Швидке сортування працює за O(n log n) у середньому та $O(n^2)$ у найгіршому випадку (наприклад, для вже відсортованого масиву при виборі a[l] як опорного), є нестійким, але використовує лише O(log n) пам'яті стеку.

Посилання на GitHub: https://github.com/TserkovniyAlex/Lab2-Sorting.git